Case 01 Answer Key

Automation Brief — Case 1 / Meera

Why

Pain: 60 voice notes unused, 40 abandoned drafts, 11 weeks without a post. Known business impact is significant - 47K impressions, wholesale enquiries from a single post.

User: Meera Pillai, founder of Skinstinct.

Outcome: Three publishable LinkedIn posts per week, under 15 minutes of Meera's active time.

Journey Today

1. Meera drops a note into her personal Telegram channel.
2. The note sits in the channel - no system processes it.
3. Occasionally she opens Google Drive, finds an old note or draft, starts writing.
4. She writes two lines, decides it's not good enough, closes the doc.
5. The draft joins forty others in the abandoned folder.
6. Nothing gets published.

Nine Checks

Check	Your evidence from the case	Pass / Fail
Kill Switches — any No = do not build		
01 **Problem Real** *Is there documented, ongoing evidence of pain?*	60 unused voice notes over 8 months, 40 abandoned drafts, 11 weeks without a post. Meera's last post drove 340 profile visits and 3 wholesale enquiries. The gap between capture and publish is documented, ongoing, and has a measurable commercial cost.	**YES**
02 **Workflow Repeated** *Does this task recur regularly?*	Meera drops 2–3 notes per week - a habit sustained over 8 months. The failure loop (open doc, write 2 lines, abandon) repeats across 40 drafts. This is not a one-off project.	**YES**
03 **Input Available** *Do all inputs exist and are they accessible now?*	Telegram channel is active. The notes/ folder has 60 fragments. The published/ folder has 15 pieces for voice reference. All inputs exist before the build starts.	**YES**
Sizing — any No = build something smaller		
04 **Output Valuable** *Would someone pay for or act on this output?*	47,000 impressions from 4 posts. One post alone drove 340 profile visits and 3 wholesale enquiries. 6,200 followers are waiting. A draft that Meera approves and publishes has demonstrated, measurable commercial value.	**YES**
05 **Impact Measurable**	Baseline: ~0.5 posts/month over 8 months. Target: 3 posts/week. Time baseline: 90–180 min stall time per post.	**YES**

Check	Your evidence from the case	Pass / Fail
Can you track improvement with a number?	Target: 15 min/week. Both post frequency and time saved are trackable from day one.	
08 **ROI Worth It** *Does the benefit clearly outweigh build and running cost?*	Three wholesale enquiries from one post represent significant revenue. Build replaces hours of weekly stall time. One converted enquiry likely exceeds build and running cost. ROI is clear.	**YES**
Boundary — any No = build the tool, human stays here		
06 **Failure Risk OK** *If the AI gets it wrong, what is the consequence?*	No output reaches LinkedIn without Meera's approval. The worst failure is a bad draft she rejects — not a post that goes live under her name. Failure consequence is low.	**YES**
07 **Judgment Protected** *Does a human review before anything consequential happens?*	Meera passed on two consultants who built end-to-end tools. She wants to remain the author of everything published. Auto-scheduling removes her judgment at the most consequential step. ← THE CUT	**NO**
09 **Owner Clear** *Is one person accountable for the output?*	Meera reviews, edits, and publishes herself. No ambiguity about who is responsible for what goes out.	**YES**

The Cut

Check killed: 07 — Judgment Protected

CASE 1 · MEERA — EXECUTION PLAN

L3 Sep 23 + B1 Sep 25

WHAT YOU'RE BUILDING

By end of L3: Meera sends a note to her Telegram bot. A draft in her voice comes back.
By end of B1: The pipeline screens notes before drafting, adds a news angle, and stores everything for Meera to approve.

THE STACK

Tool	Role
Claude (browser)	Builds the Voice Skill — no code, before anything else
Claude Code	Where the project lives and gets built
Gemini	Reads the Voice Skill and writes the draft
Telegram	Where Meera sends notes and receives drafts
Vercel	Puts the project online
GitHub	Saves progress between sessions

BEFORE THE BUILD · MINS 0–25

- **0–5** · Read the brief. Find the Cut before it's said.
- **5–20** · Components map, box by box. Box 3 (Voice Skill) gets the most time. Keep Claude open — you'll use it in the next step.
- **20–25** · Demo: Mihirr runs the finished pipeline. A note goes in, a draft comes back to Telegram. Then a short overview of what you're building in the next 35 minutes.

CHECKPOINT L3·1 · 10 MIN · THE VOICE SKILL

Done by minute 35. No code yet. You need: Claude open in your browser, the `published/` folder from your dataset.

- Open a new Claude conversation
- Copy everything from the `published/` folder — all 4 LinkedIn posts and all 11 newsletters — and paste it all into Claude

- Send this message:
  "Based only on the writing I've shared, describe this founder's voice in 150–200 words. Cover: how she opens, her sentence length and rhythm, how she uses data and evidence, what she never says, and how personal she gets. Be specific — refer to what's actually in the text."
- Read the response. If anything feels generic, ask Claude to point to a specific example from the writing and be more precise
- Once satisfied, ask Claude:
  "Save this voice profile as a plain text file called `voice-skill.txt` *so I can download it."*
- Download the file to your computer

DONE WHEN: `voice-skill.txt` is on your computer and reads as something specific to Meera, not generic writing advice.

CHECKPOINT L3·2 · 17 MIN · BUILD THE PROJECT

Done by minute 52. You need: Claude Code open, your `voice-skill.txt` *file, your Telegram bot token, your Gemini API key.*

Set up the project

- Open Claude Code. Start a fresh session
- Send this as your first message — this tells Claude Code everything it needs to know to build the project:
  "I'm building a tool for a founder called Meera. She sends notes as text messages to a Telegram bot. The bot takes each note, sends it to Gemini along with instructions on how she writes, and sends the draft post back to her in the same Telegram chat. The project will be hosted on Vercel. Set up a complete project for me with all the files needed to make this work. I'll add my API keys and the voice instructions separately."
- Read through what Claude Code produces before moving forward. You should be able to follow what each file does even if you didn't write it

Add your keys

- Find the `.env` file in the project. It will have placeholder values like `your_token_here`
- Replace them with your actual keys:
  - `TELEGRAM_BOT_TOKEN` — from BotFather
  - `GEMINI_API_KEY` — from Google AI Studio
- This file is private and must never be pushed to GitHub. Check that `.env` is listed in `.gitignore` — if it isn't, tell Claude Code to add it

Add the Voice Skill

- Upload `voice-skill.txt` into the Claude Code project
- Once uploaded, send this message:
  "I've uploaded a file called `voice-skill.txt`*. It describes how this founder writes. Update the project so that every time a draft is generated, Gemini receives this file's contents as the instruction for how to write the post."*
- Ask Claude Code to show you where in the code the voice profile is being used so you can confirm it's in the right place

Publish the project

- Push to GitHub
- Open Vercel, import the project from GitHub
- Before deploying, go to Environment Variables and add both keys — exactly as they appear in your `.env` file
- Deploy and wait for it to finish
- Copy your live project URL — it looks like `https://your-project.vercel.app`

DONE WHEN: Vercel shows the project is live and you have your URL ready.

CHECKPOINT L3·3 · 8 MIN · CONNECT AND TEST

Done by minute 60. You need: your bot token, your Vercel URL.

Connect Telegram to your project

- Right now Telegram doesn't know your project exists. This step tells Telegram: "whenever someone sends a message to this bot, deliver it to this address."
- Open a new browser tab and paste the following address, replacing the two placeholders with your own details:

```
https://api.telegram.org/bot[YOUR_BOT_TOKEN]/setWebhook?url=[YOUR_VERCEL_URL]/api/webhook
```

Example of what it looks like complete:

```
https://api.telegram.org/bot7412938:AABcd.../setWebhook?url=https://your-project.vercel.app/api/webhook
```

- Press Enter. You should see `"ok":true` in the browser. If not, check that your bot token has no extra spaces and your Vercel URL is from a completed deployment, not a preview

Test

- Pick a strong note from your dataset — one with a clear point and enough substance to work with
- Send it as a message to your capture channel on Telegram
- Wait a few seconds. The draft should arrive in the same channel
- Read it. Does it sound like Meera? If it reads like generic LinkedIn content, the voice profile isn't reaching Gemini — ask Claude Code to check that the contents of `voice-skill.txt` are being passed through correctly

Before leaving the room: push to GitHub. B1 starts from what you pushed.

B1 · BUILD SESSION · SEP 25 · 2 HOURS

CHECKPOINT B1·1 · 30 MIN · SCORING

Right now every note gets drafted — the logistics reminders, the abandoned half-sentences, all of it. This step adds a score before the draft is generated.

- Add a step before drafting where Gemini reads the note and gives it a score from 0–10 with a one-line reason
- If the score is below 6: the pipeline sends a short message back to Telegram explaining why no draft was made, and stops
- If the score is 6 or above: drafting continues as normal
- Test with a strong, substantive note — should score 6 or above and produce a draft
- Test with a task reminder or abandoned thought — should score 3 or below, no draft, rejection message sent

DONE WHEN: two contrasting notes produce the right outcomes. If everything passes, the scoring prompt is too lenient.

CHECKPOINT B1·2 · 45 MIN · NEWS ANGLE

After a note passes scoring, the pipeline finds a relevant news article before writing the draft.

- **Extract keywords** — ask Gemini to pull 3–5 search terms from the approved note and return a short search phrase
- **Fetch news** — use that phrase to search Google News. No account needed, no key needed. Pull the top result: headline, source, date, and a one-line summary
- **Update the draft** — pass the news item to Gemini alongside the note and Voice Skill, with this instruction: *"If this news item is genuinely relevant, use it to make the post timely. If it doesn't fit naturally, ignore it."*
- **Add the verify flag** — any draft that uses a news item must include this at the end:

```
————————————————————————————
NEWS SOURCE: [headline]
FROM: [publication] · [date]
LINK: [url]
⚠ Check this before publishing — you are the author of this claim
————————————————————————————
```

This is not optional. A fact published in Meera's name that she hasn't checked is the exact failure the pipeline exists to prevent.

DONE WHEN: a draft arrives in Telegram with a news angle, source, date, link, and verify flag all visible.

CHECKPOINT B1·3 · 30 MIN · MEMORY

Right now nothing is saved. This step creates the memory layer.

- Set up three tables in Supabase: one for notes, one for drafts, one for the voice skill
- After every run: the note is saved, the draft is saved with a status of `pending`
- After Meera replies `APPROVE` or `REJECT` in Telegram: the draft status updates
- Rejected notes and rejected drafts are kept, not deleted — they show what needs improving

DONE WHEN: send a note, check Supabase — the note and draft are there. Reply APPROVE, refresh — the status has updated and stays updated.

FINAL 15 MIN · MODEL COMPARISON

- Swap the Gemini drafting call to Claude
- Run the same note through both
- Read the two drafts side by side and write down in one sentence what changed

This is the reason the stack separates scoring (Gemini Flash, fast and cheap) from drafting (Claude, better at holding a voice across a full post).

WHAT TO USE WHEN

Task	Tool	Why
Building the Voice Skill	Claude (browser)	One session, no code
Scoring notes	Gemini Flash	Fast, cheap, no judgment needed
Finding keywords	Gemini Flash	Fast, cheap, mechanical
Fetching news	Google News (free)	No key, no account
Writing drafts — L3	Gemini	Available, fast enough for the demo
Writing drafts — B1 onwards	Claude	Holds voice better across a full post
Saving everything	Supabase	Free, persistent
Hosting the project	Vercel	Already familiar, free

Case 1 · Meera / Content Engine — Components Map · Answer Key

